

ZeitTracker – Deployment Guide

Complete guide for packaging, deploying, and maintaining ZeitTracker in a company network.

Table of Contents

- 1. [Prerequisites](#)
- 2. [Architecture Overview](#)
- 3. [Prepare the Server](#)
- 4. [Transfer the Project](#)
- 5. [Configure Environment](#)
- 6. [Build & Start](#)
- 7. [HTTPS with Reverse Proxy](#)
- 8. [DNS / Internal Hostname](#)
- 9. [Firewall & Network](#)
- 10. [Backup & Restore](#)
- 11. [Updates & Redeployment](#)
- 12. [Monitoring & Logs](#)
- 13. [Troubleshooting](#)
- 14. [Security Checklist](#)
- 15. [Quick Reference](#)

1. Prerequisites

On the Server

Software	Minimum Version	Check Command
Docker Engine	24.0+	<code>docker --version</code>
Docker Compose	2.20+ (V2)	<code>docker compose version</code>
Linux (recommended)	Ubuntu 22.04 / Debian 12 / RHEL 9	<code>cat /etc/os-release</code>

Windows Server: Docker Desktop or Docker Engine via WSL2 works, but Linux is recommended for production.

Install Docker (Ubuntu/Debian)

```
# Install Docker
curl -fsSL https://get.docker.com | sh

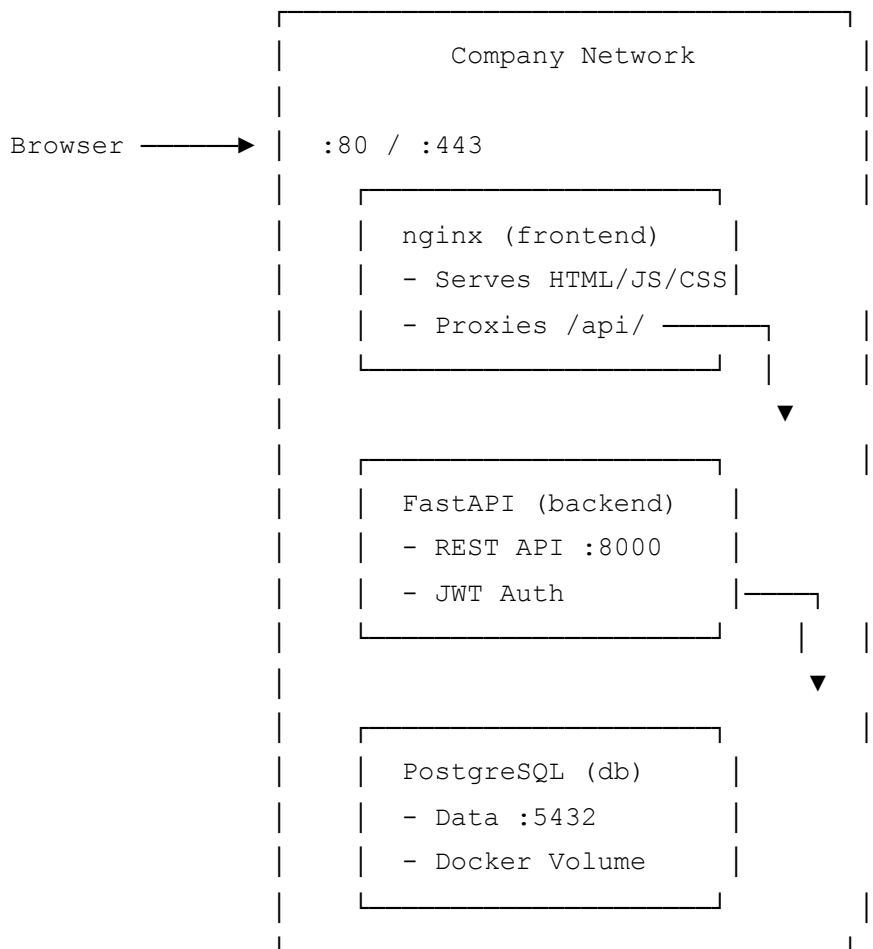
# Add your user to the docker group (log out & back in after)
sudo usermod -aG docker $USER

# Verify
docker compose version
```

Hardware Recommendations

Users	CPU	RAM	Disk
1–20	2 cores	2 GB	10 GB
20–100	4 cores	4 GB	20 GB
100–500	4 cores	8 GB	50 GB

2. Architecture Overview



All three services run as Docker containers, connected via an internal Docker network (`zeittracker`). Only the frontend port (80 or 443) needs to be exposed externally.

3. Prepare the Server

```
# Create a dedicated directory
sudo mkdir -p /opt/zeittracker
sudo chown $USER:$USER /opt/zeittracker
```

4. Transfer the Project

Option A: Git Clone (if you have an internal Git repo)

```
cd /opt/zeittracker
git clone https://your-git-server.com/zeittracker.git .
```

Option B: Copy from Dev Machine

From your **development machine**, package and transfer:

```
# On your dev machine - create a tarball (excludes unnecessary files)
cd /path/to/Time-Tracker-Simple
tar czf zeittracker.tar.gz \
  --exclude='.git' \
  --exclude='__pycache__' \
  --exclude='.pytest_cache' \
  --exclude='node_modules' \
  --exclude='backups' \
  --exclude='.env' \
  .

# Transfer to server (replace with your server IP/hostname)
scp zeittracker.tar.gz user@server:/opt/zeittracker/
```

On the **server**:

```
cd /opt/zeittracker
tar xzf zeittracker.tar.gz
rm zeittracker.tar.gz
```

Option C: Pre-built Docker Images (air-gapped network)

If the server has **no internet access**, export the images on your dev machine:

```
# On dev machine - build first
cd /path/to/Time-Tracker-Simple
docker compose build

# Save images to files
docker save time-tracker-simple-backend:latest | gzip > backend-image.tar.gz
docker save time-tracker-simple-frontend:latest | gzip > frontend-image.tar.gz
docker save postgres:16-alpine | gzip > postgres-image.tar.gz

# Transfer all three files + docker-compose.yml + .env to the server
scp backend-image.tar.gz frontend-image.tar.gz postgres-image.tar.gz \
    docker-compose.yml .env user@server:/opt/zeittracker/
```

On the **server**:

```
cd /opt/zeittracker
gunzip -c backend-image.tar.gz | docker load
gunzip -c frontend-image.tar.gz | docker load
gunzip -c postgres-image.tar.gz | docker load
rm *.tar.gz
```

5. Configure Environment

Create the `.env` file

```
cd /opt/zeittracker
cp .env.example .env      # or create from scratch
nano .env                 # edit with your values
```

.env — Full Configuration

```
# =====  
# ZeitTracker Configuration  
# =====  
  
# — PostgreSQL —  
POSTGRES_USER=zeittracker  
POSTGRES_PASSWORD=<GENERATE_A_STRONG_PASSWORD>  
POSTGRES_DB=zeittracker  
  
# — Backend —  
# Must match POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB above  
DATABASE_URL=postgresql+asyncpg://zeittracker:<SAME_PASSWORD>@db:5432/zeittracker  
  
# JWT Secret - generate with: openssl rand -hex 32  
JWT_SECRET=<GENERATE_WITH_openssl_rand_-hex_32>  
  
# Initial admin account (created on first startup)  
ADMIN_EMAIL=admin@yourcompany.com  
ADMIN_PASSWORD=<STRONG_INITIAL_PASSWORD>  
  
# CORS - set to your actual domain/IP  
CORS_ORIGINS=https://zeit.yourcompany.com,http://zeit.yourcompany.com  
  
# Timezone  
TZ=Europe/Berlin  
  
# Default vacation days per employee per year  
DEFAULT_VACATION_DAYS=30
```

Generate Secure Values

```
# Generate a strong database password  
openssl rand -base64 24  
  
# Generate JWT secret  
openssl rand -hex 32
```

Important: Use the **same password** in `POSTGRES_PASSWORD` and in the `DATABASE_URL` connection string.

Production Hardening in `docker-compose.yml`

For production, remove the test volume mounts and don't expose the database port externally. Edit `docker-compose.yml` :

```
db:
  # ...
  ports: []          # ← Remove "5432:5432" or delete the ports section entirely

backend:
  # ...
  ports: []          # ← Remove "8000:8000" (nginx proxies to it internally)
  volumes: []        # ← Remove the test volume mounts
```

Only the frontend needs an exposed port:

```
frontend:
  ports:
    - "80:80"        # or "443:443" if using HTTPS directly
```

6. Build & Start

```
cd /opt/zeittracker

# Build and start all services
docker compose up -d --build

# Verify everything is running
docker compose ps
```

Expected output:

NAME	STATUS	PORTS
time-tracker-simple-db-1	Up (healthy)	
time-tracker-simple-backend-1	Up	
time-tracker-simple-frontend-1	Up	0.0.0.0:80->80/tcp

Verify it Works

```
# Check backend health
curl http://localhost/health

# Check the web UI
curl -s http://localhost/ | head -5
```

Open `http://<server-ip>` in a browser. Log in with the `ADMIN_EMAIL` / `ADMIN_PASSWORD` from your `.env`.

First Steps After Login

1. **Change the admin password** immediately (Profile → Change Password)
2. **Create departments** (Verwaltung → Abteilungen)
3. **Create users** and assign them to departments
4. The app is ready to use

7. HTTPS with Reverse Proxy

For production, **always use HTTPS**. Two common approaches:

Option A: External Reverse Proxy (Recommended for Company Networks)

If your company already has a reverse proxy (e.g., Apache, IIS, HAProxy, Traefik), point it to the ZeitTracker frontend on port 80.

Example: Nginx on the host (separate from the container)

```
sudo apt install nginx certbot python3-certbot-nginx
```

Create `/etc/nginx/sites-available/zeittracker`:


```

server {
    listen 80;
    server_name zeit.yourcompany.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name zeit.yourcompany.com;

    ssl_certificate      /etc/ssl/certs/zeit.yourcompany.com.crt;
    ssl_certificate_key  /etc/ssl/private/zeit.yourcompany.com.key;

    # Modern TLS settings
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    client_max_body_size 50M;

    location / {
        proxy_pass http://127.0.0.1:80;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
    }
}

```

```

sudo ln -s /etc/nginx/sites-available/zeittracker /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

```

Company SSL certificate: Your IT department likely has internal CA certificates. Place the `.crt` and `.key` files as shown above.

Option B: Let's Encrypt (Public-Facing Servers)

```
sudo certbot --nginx -d zeit.yourcompany.com
```

Update CORS After HTTPS

In `.env`, update:

```
CORS_ORIGINS=https://zeit.yourcompany.com
```

Then restart the backend:

```
docker compose up -d --build backend
```

8. DNS / Internal Hostname

Ask your IT department to create a DNS A record:

Type	Name	Value
A	zeit.yourcompany.com	192.168.x.x

If no DNS is available, users can access via IP: `http://192.168.x.x`

For testing, users can add to their local `hosts` file:

```
192.168.x.x    zeit.yourcompany.com
```

9. Firewall & Network

Required Ports

Port	Direction	Purpose	Who Needs Access
80	Inbound	HTTP (or redirect)	All employees
443	Inbound	HTTPS	All employees
22	Inbound	SSH (admin only)	IT administrators only
5432	NONE	PostgreSQL	✗ Do NOT expose
8000	NONE	Backend API	✗ Do NOT expose

UFW (Ubuntu Firewall)

```
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP
sudo ufw allow 443/tcp     # HTTPS
sudo ufw enable
```

Ports 5432 and 8000 stay **internal to Docker** – never open them to the network.

10. Backup & Restore

Automatic Daily Backups

A backup script (`backup.sh`) is included. Set up a daily cron job:

```
# Edit crontab
crontab -e

# Add this line (daily at 2:00 AM, keep 30 days)
0 2 * * * /opt/zeittracker/backup.sh 30 >> /var/log/zeittracker-backup.log 2>&1
```

Manual Backup

```
cd /opt/zeittracker  
./backup.sh
```

Backups are saved to `./backups/zeittracker_YYYYMMDD_HHMMSS.sql.gz` .

Restore from Backup

```
# Stop the backend first  
docker compose stop backend  
  
# Restore (replace with actual backup filename)  
gunzip -c backups/zeittracker_20260216_020000.sql.gz | \  
    docker exec -i time-tracker-simple-db-1 \  
    psql -U zeittracker -d zeittracker  
  
# Restart  
docker compose up -d backend
```

App-Level Backup (JSON via Admin Panel)

Admins can also export/import data through the web UI:

- **Verwaltung** → **Backup** → Export/Import full database as JSON
- **Daten** tab → Export/Import individual user data as JSON

Backup Best Practices

- Store backups on a **separate drive or network share**
- Test restoring from backup **at least once** before going live
- Keep offsite copies for disaster recovery

11. Updates & Redeployment

Update from Git

```
cd /opt/zeittracker
git pull origin main
docker compose up -d --build
```

Update from Tarball

```
cd /opt/zeittracker
docker compose down
tar xzf zeittracker-update.tar.gz
docker compose up -d --build
```

Update from Pre-built Images (Air-Gapped)

```
cd /opt/zeittracker
docker compose down
gunzip -c backend-image.tar.gz | docker load
gunzip -c frontend-image.tar.gz | docker load
docker compose up -d
```

Rollback

If an update breaks things:

```
# Check logs
docker compose logs backend --tail 50

# Rollback to previous source code
git checkout <previous-commit>
docker compose up -d --build
```

Zero-Downtime Update

```
# Rebuild without stopping
docker compose up -d --build --no-deps backend
docker compose up -d --build --no-deps frontend
```

12. Monitoring & Logs

View Logs

```
# All services
docker compose logs -f

# Specific service
docker compose logs -f backend
docker compose logs -f frontend
docker compose logs -f db

# Last 100 lines
docker compose logs --tail 100 backend
```

Health Check

```
# Backend health endpoint
curl http://localhost/health

# Container status
docker compose ps

# Resource usage
docker stats --no-stream
```

Log Rotation

Docker logs can grow large. Add to `/etc/docker/daemon.json` :

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

Then restart Docker: `sudo systemctl restart docker`

13. Troubleshooting

Container won't start

```
# Check logs for errors
docker compose logs backend --tail 50
```

```
# Common: database not ready yet → backend auto-retries via depends_on healthcheck
# Common: wrong DATABASE_URL in .env
```

Can't connect from browser

```
# Check if frontend is running
docker compose ps frontend
```

```
# Check if port 80 is open
ss -tlnp | grep :80
```

```
# Check firewall
sudo ufw status
```

Database connection errors

```
# Check DB is healthy
docker compose exec db pg_isready -U zeittracker

# Check DATABASE_URL matches POSTGRES_USER/PASSWORD/DB in .env
```

"502 Bad Gateway"

The backend hasn't started yet or crashed:

```
docker compose logs backend --tail 20
docker compose restart backend
```

Reset Admin Password

```
# Update .env with new ADMIN_PASSWORD, then:
docker compose exec backend python -c "
import asyncio
from app.database import async_session_factory
from app.models.user import User
from app.utils.password import hash_password
from sqlalchemy import select

async def reset():
    async with async_session_factory() as db:
        result = await db.execute(select(User).where(User.email == 'admin@example.com'))
        user = result.scalar_one()
        user.password_hash = hash_password('NewPassword123!')
        await db.commit()
        print('Password reset successfully')

asyncio.run(reset())
"
```

Complete Reset (Delete All Data)

```
docker compose down -v      # ⚠️ Deletes the database volume!
docker compose up -d --build
```


14. Security Checklist

Before going live, verify:

- ☐ Changed `POSTGRES_PASSWORD` to a strong random value
- ☐ Changed `JWT_SECRET` to a random 64+ character hex string
- ☐ Changed `ADMIN_PASSWORD` to a strong password
- ☐ Changed the admin password again **via the web UI** after first login
- ☐ `CORS_ORIGINS` set to the actual production URL only
- ☐ Port 5432 (PostgreSQL) is **not** exposed externally
- ☐ Port 8000 (backend) is **not** exposed externally
- ☐ HTTPS is enabled (via reverse proxy or directly)
- ☐ Removed `tests` volume mount from `docker-compose.yml`
- ☐ Automatic backups are configured (cron job)
- ☐ Backup restore has been tested
- ☐ Firewall only allows ports 80, 443, and 22
- ☐ `.env` file has restricted permissions (`chmod 600 .env`)

15. Quick Reference

Daily Operations

```
# Start
docker compose up -d

# Stop
docker compose down

# Restart a service
docker compose restart backend

# View status
docker compose ps

# View logs
docker compose logs -f backend
```

Maintenance

```
# Run backup
./backup.sh

# Run tests (in container)
docker compose exec backend python -m pytest tests/ -v

# Update and redeploy
git pull && docker compose up -d --build

# Clean unused Docker resources
docker system prune -f
```

Emergency

```
# Stop everything
docker compose down

# Full reset (⚠ data loss!)
docker compose down -v && docker compose up -d --build

# Restore from backup
docker compose stop backend
gunzip -c backups/zeittracker_YYYYMMDD.sql.gz | \
    docker exec -i time-tracker-simple-db-1 psql -U zeittracker -d zeittracker
docker compose up -d backend
```

Summary: Minimum Steps to Deploy

```
# 1. Install Docker on the server
curl -fsSL https://get.docker.com | sh

# 2. Copy the project to the server
scp -r Time-Tracker-Simple/ user@server:/opt/zeittracker/

# 3. Configure
cd /opt/zeittracker
nano .env    # Set passwords, JWT secret, CORS, admin email

# 4. Harden docker-compose.yml (remove ports 5432 & 8000, remove test volumes)

# 5. Start
docker compose up -d --build

# 6. Open http://<server-ip> and log in

# 7. Set up HTTPS + DNS + daily backups
```

That's it. The application is self-contained — no external dependencies beyond Docker.